Algorithms and Data Structures

26.02.2024

Exercise 1 (4.0 points)
Given an array of 2^n elements, the following function partitions it into sub-arrays of half the size until all sub-arrays have a length equal to one cell.

```
void show (int v[], int l, int r) {
  int i, c;
  if (l >= r) {
    return;
  }
  c = (r+l)/2;
  show (v, l, c;
  show (v, c+1, r);
  return;
}
```

Write the recurrence equation of the recursive function, solve it through unfolding and substitutions, and express its asymptotic complexity.

Solution
Please see theacher's overhead for a complete analysis of the topic.

Exercise 2 (1.5 points)
Consider a binary tree whose visits return the following sequences.

```
Pre-order:      A   B   I   E   F   H   K   J   C   D   G
In-order:       I   B   E   H   K   F   J   A   D   C   G
Post-order:     I   K   H   J   F   E   B   D   G   C   A
```

Report the sequence of keys stored on the tree leaves, moving on the tree from left to right. Please report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. The following is an example of the response format: A X C Y etc.

Solution

```
### Tree guess
Node Key  Left Right
   1    A    2    3
   2    B    4    5
   3    C    6    7
   4    I NULL NULL
   5    E NULL    8
   6    D NULL NULL
   7    G NULL NULL
   8    F    9   10
   9    H NULL   11
  10    J NULL NULL
  11    K NULL NULL
```

Response
I K J D G

Exercise 3 (1.5 points)
Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to `M=23`. Insertions occur character by character using open addressing with double hashing. Use functions $h_1(k) = k\%M$ and $h_2(k) = 1 + (k\%97)$. Each character is identified by its index in the English alphabet (i.e., `A=1`, ..., `Z=26`). Equal letters are identified by a different subscript (i.e., `A` and `A` become `A1` and `A2`).

```
C I N C I N
```

Indicate in which elements the last three letters of the sequence are placed, i.e., `C`, `I`, and `N`, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Solution

```
### hash-table
Character keys
Hash table size = 23
Number of keys  =  6
Double hashing.
Key:  C I N C I N
Key=C k=003 Final=03
```

```
Key=I k=009 Final=09
Key=N k=014 Final=14
Key=C k=003 Coll.= 3 Final=07
Key=I k=009 Coll.= 9 Final=19
Key=N k=014 Coll.=14 Final=06
00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22
-  -  -  C  -  -  N  C  -  I  -  -  -  -  N  -  -  -  -  I  -  -  -
```

Response
7 19 6

Exercise 4 (1.5 points)
Suppose we have an initially empty priority queue implemented with a maximum heap. Consider the following sequence of integers and "*" characters, where each integer corresponds to one insertion into the priority queue, and each character "*" corresponds to one extraction.

```
 3  7 11 13  4  7 *  *
```

Report the sequence of values as they are stored in the array representing the priority queue at the end of the entire process. Please show the entire array content as a sequence of integer values separated by a single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8 etc.

Solution

```
### Heap insertion and extraction
Max-Heap.
Insert keys:  3  7 11 13  4  7 *  *
Insert    3:  3
Insert    7:  7  3
Insert   11: 11  3  7
Insert   13: 13 11  7  3
Insert    4: 13 11  7  3  4
Insert    7: 13 11  7  3  4  7
Extract  13: 11  7  7  3  4
Extract  11:  7  4  7  3
```

Response
7 4 7 3

Exercise 5 (1.5 points)
The following capital letters are given with their absolute frequency.

```
A:11 B:14 C:7 D:5 E:3 F:9
```

Find an optimal Huffman code for all symbols in the set using a greedy algorithm. Indicate which one of the following encoding is correct.

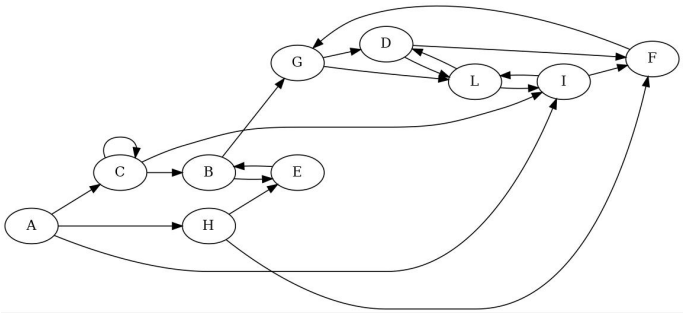
1. A(01) B(10) C(110) D(1111) E(1110) F(00)
2. A(010) B(101) C(011) D(0011) E(0000) F(000)
3. A(00) B(01) C(10) D(110) E(1110) F(1111)
4. A(01) B(10) C(110) D(1110) E(1111) F(001)
5. A(001) B(100) C(110) D(1111) F(00) E(1110)
6. A(001) B(100) C(110) D(11111) E(1110) F(00)
7. A(010) B(100) C(110) D(11110) E(11110) F(00)
8. A(001) B(100) C(1110) D(1111) E(11110) F(000)

Solution

```
### Huffman
Huffman:
Number of codes:  6
A:11 B:14 C:7 D:5 E:3 F:9
Merging: {E}(3)[-1][-1] + {D}(5)[-1][-1] = {ED}(8)[0][1]
Merging: {C}(7)[-1][-1] + {ED}(8)[0][1] = {CED}(15)[2][3]
Merging: {F}(9)[-1][-1] + {A}(11)[-1][-1] = {FA}(20)[4][5]
Merging: {B}(14)[-1][-1] + {CED}(15)[2][3] = {BCED}(29)[6][7]
Merging: {FA}(20)[4][5] + {BCED}(29)[6][7] = {FABCED}(49)[8][9]
F 00
A 01
B 10
C 110
E 1110
D 1111
```

Response
1

Exercise 6 (1.5 points)
Visit the following graph depth-first, starting at node A. Label nodes with discovery and end-processing times. Start with the discovery time set to 1 on A. When necessary, consider nodes and edges in alphabetic order.



Display the end-processing time of all vertices. Please indicate the end-processing time of all vertices sorted in alphabetic order (i.e., display the end-processing time for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 15 13 2 16 8 etc.

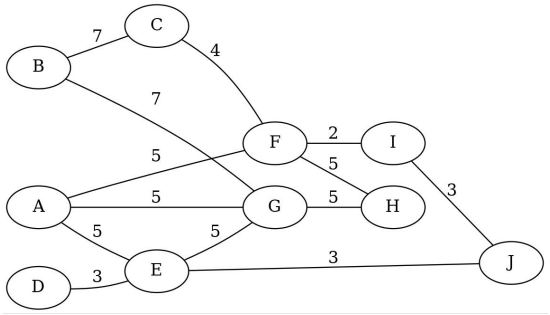
Solution

```
Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] L
Adjacency Matrix:
 0 0 1 0 0 0 0 1 1 0
 0 0 0 0 1 0 1 0 0 0
 0 1 1 0 0 0 0 0 1 0
 0 0 0 0 0 1 0 0 0 1
 0 1 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 1 0 0 0
 0 0 0 1 0 0 0 0 0 1
 0 0 0 0 1 1 0 0 0 0
 0 0 0 0 0 1 0 0 0 1
 0 0 0 1 0 0 0 0 1 0
### Graph visit
BFS:
Node=[ 0] A Discovery_time= 0 Predecessor=[-1]
Node=[ 1] B Discovery_time= 2 Predecessor=[ 2]
Node=[ 2] C Discovery_time= 1 Predecessor=[ 0]
Node=[ 3] D Discovery_time= 3 Predecessor=[ 9]
Node=[ 4] E Discovery_time= 2 Predecessor=[ 7]
Node=[ 5] F Discovery_time= 2 Predecessor=[ 7]
Node=[ 6] G Discovery_time= 3 Predecessor=[ 1]
Node=[ 7] H Discovery_time= 1 Predecessor=[ 0]
Node=[ 8] I Discovery_time= 1 Predecessor=[ 0]
Node=[ 9] L Discovery_time= 2 Predecessor=[ 8]
DFS:
List of edges:
[ 0] A -> [ 2] C : (T) Tree
[ 2] C -> [ 1] B : (T) Tree
[ 1] B -> [ 4] E : (T) Tree
[ 4] E -> [ 1] B : (B) Back
[ 1] B -> [ 6] G : (T) Tree
[ 6] G -> [ 3] D : (T) Tree
[ 3] D -> [ 5] F : (T) Tree
[ 5] F -> [ 6] G : (B) Back
[ 3] D -> [ 9] L : (T) Tree
[ 9] L -> [ 3] D : (B) Back
[ 9] L -> [ 8] I : (T) Tree
[ 8] I -> [ 5] F : (C) Cross
[ 8] I -> [ 9] L : (B) Back
[ 6] G -> [ 9] L : (F) Forward
[ 2] C -> [ 2] C : (B) Back
[ 2] C -> [ 8] I : (F) Forward
[ 0] A -> [ 7] H : (T) Tree
[ 7] H -> [ 4] E : (C) Cross
[ 7] H -> [ 5] F : (C) Cross
[ 0] A -> [ 8] I : (F) Forward
List of vertices:
[ 0] A: dist_time= 1 endp_time=20 pred=[-1] -
[ 1] B: dist_time= 3 endp_time=16 pred=[ 2] C
```

```
[ 2] C: dist_time= 2 endp_time=17 pred=[ 0] A
[ 3] D: dist_time= 7 endp_time=14 pred=[ 6] G
[ 4] E: dist_time= 4 endp_time= 5 pred=[ 1] B
[ 5] F: dist_time= 8 endp_time= 9 pred=[ 3] D
[ 6] G: dist_time= 6 endp_time=15 pred=[ 1] B
[ 7] H: dist_time=18 endp_time=19 pred=[ 0] A
[ 8] I: dist_time=11 endp_time=12 pred=[ 9] L
[ 9] L: dist_time=10 endp_time=13 pred=[ 3] D
Reachable nodes from vertex [ 0] A:
[ 0] A - [ 1] B - [ 2] C - [ 3] D - [ 4] E - [ 5] F - [ 6] G - [ 7] H - [ 8] I - [ 9] L -
UN-reachable nodes from vertex [ 0] A:
none
```

Response
20 16 17 14 5 9 15 19 12 13

Exercise 7 (1.5 points)
Given the following undirected and weighted graph, find a minimum spanning tree using Kruskal's algorithm.



Indicate the total weight of the final minimum spanning tree. Report one single integer value. No other symbols must be included in the response. The following is an example of the response format: 13

Solution

```
UN-directed matrix ... stated as such.
Weighted matrix ... stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J
Adjacency Matrix:
 0 0 0 0 5 5 5 0 0 0
 0 0 7 0 0 0 7 0 0 0
 0 7 0 0 0 4 0 0 0 0
 0 0 0 0 3 0 0 0 0 0
 5 0 0 3 0 0 5 0 0 3
 5 0 4 0 0 0 0 5 2 0
 5 7 0 0 5 0 0 5 0 0
 0 0 0 0 0 5 5 0 0 0
 0 0 0 0 0 2 0 0 0 3
 0 0 0 0 3 0 0 0 3 0
### Graph MST (Kruskal and Prim)
Kruskal
Sorted array of edges: src dst weight:
                        F  I   2
                        D  E   3
                        E  J   3
                        I  J   3
                        C  F   4
                        A  E   5
                        A  F   5
                        A  G   5
                        E  G   5
                        F  H   5
                        G  H   5
                        B  C   7
                        B  G   7
MST (list of edges):
Edge [ 5]  F - [ 5]  I ==> weight =  2
Edge [ 3]  D - [ 3]  E ==> weight =  3
Edge [ 4]  E - [ 4]  J ==> weight =  3
Edge [ 8]  I - [ 8]  J ==> weight =  3
Edge [ 2]  C - [ 2]  F ==> weight =  4
Edge [ 0]  A - [ 0]  E ==> weight =  5
Edge [ 0]  A - [ 0]  G ==> weight =  5
Edge [ 5]  F - [ 5]  H ==> weight =  5
Edge [ 1]  B - [ 1]  C ==> weight =  7
Total tree weight                  = 37
Prim
MST (list of edges):
Edge [ 0]  A - [ 4]  E ==> weight =  5
Edge [ 4]  E - [ 3]  D ==> weight =  3
```

```
Edge [ 4]  E - [ 9]  J ==> weight =  3
Edge [ 9]  J - [ 8]  I ==> weight =  3
Edge [ 8]  I - [ 5]  F ==> weight =  2
Edge [ 5]  F - [ 2]  C ==> weight =  4
Edge [ 0]  A - [ 6]  G ==> weight =  5
Edge [ 5]  F - [ 7]  H ==> weight =  5
Edge [ 2]  C - [ 1]  B ==> weight =  7
Total tree weight                  = 37
```

Response
37

Exercise 8 (2.0 points)
Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```
#define R 4
#define C 5
int main(void) {
  int mat[R][C] ={{ 3,  6, 1, 0, 9},
                  {12,  5, 9, 7, 2},
                  {13,  5, 9, 6, 2},
                  { 1, 14, 6, 9, 5}};
  int i, j, v1, v2;
  for (i=0; i<R; i++) {
    for (j=0; j<C; j++) {
      if (j==0) {
        v1=v2=mat[i][j];
      } else {
        if (mat[i][j]<v1) {
          v1=mat[i][j];
        } else {
          if (mat[i][j]>v2) {
            v2=mat[i][j];
          }
        }
      }
    }
    printf ("%d ", v2-v1);
  }
  return 0;
}
```

Response
9 10 11 13

Exercise 9 (2.0 points)
The following function implements a combinatorics principle.

```
int f (int *val, int *sol, int *mark, int n, int k, int count, int pos){
  int i;
  if (pos >= k){
    for (i=0; i<k; i++)
      printf("%d ", sol[i]);
    printf("\n");
    return count+1;
  }
  for (i=0; i<n; i++){
    if (mark[i] == 0) {
      mark[i] = 1;   // LINE 1
      sol[pos] = val[i];
      count = f(val,sol,mark,n,k,count,pos+1);   // LINE 2
      mark[i] = 0;   // LINE 3
    }
  }
  return count;
}
```

Indicate which one of the following statements is correct. Note that incorrect answers imply a penalty in the final score.

1. Function `f` computes simple arrangements.
2. Function `f` computes simple combinations.
3. In simple arrangements, the order matters, whereas in simple combinations, it does not.
4. In simple arrangements, the order does not matter, whereas in simple combinations, it does.
5. Simple permutation may be generated using the same function but with `k` equal to `n`.
6. Permutation with repetitions be generated using the same function but with `k` equal to `n`.

7. In Line 2, `k` must be `k+i`.
8. In Lines 1 and 3, we must write `mark[i]++` and `mark[i]--`, respectively.

Response

1. TRUE.
2. FALSE.
3. TRUE.
4. FALSE.
5. TRUE.
6. FALSE.
7. FALSE.
8. FALSE.

Exercise 10 (2.0 points)
Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```
#define R 3
#define C 4
void f1 (int [R][C], int);
void f2 (int [R][C], int);
int main (void){
  int mat[R][C] = {{1,2,3,4},
                   {5,6,7,8},
                   {9,10,11,12}};
  f1(mat,0);
  return 0;
}
void f1 (int mat[R][C], int n) {
  int i;
  if (n==C-1)
    return;
  for (i=0; i<C; i++) {
    printf ("%d ", mat[n][i]);
  }
  f2(mat,n+1);
  return;
}
void f2 (int mat[R][C], int n) {
  int i;
  if (n==C-1)
    return;
  for (i=C-1; i>=0; i--) {
    printf ("%d ", mat[n][i]);
      }
  f1(mat,n+1);
      return;
}
```

Response
1 2 3 4 8 7 6 5 9 10 11 12

Exercise 11 (3.0 points)
Write the function

```
char *merge_string (char *s1, char *s2);
```

which receives in input two strings `s1` and `s2`, including only small alphabetic characters (`a`−`z`). The function must verify that the strings include **only** alphabetical characters **sorted in ascending order**. Moreover, it must apply the merge algorithm to generate a new string including all characters of `s1` and `s2` alphabetically sorted. The function must dynamically allocate the newly generated string and return its pointer.
For example, if `s1 = ''abcdxyz''` and `s2 = ''abcdefgh''` the function must generate the string `''aabbccddefghxyz''` and return its pointer.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution
To be done.

Exercise 12 (5.0 points)
A BST has a string of undefined length as a key and no other data field. A program must manipulate a set of such BSTs in which the pointer to each BST root is stored in an array of pointers of size `N`, where `N` is a pre-defined constant.
Write the function:

```
void display_common (node_t *root[N]);
```

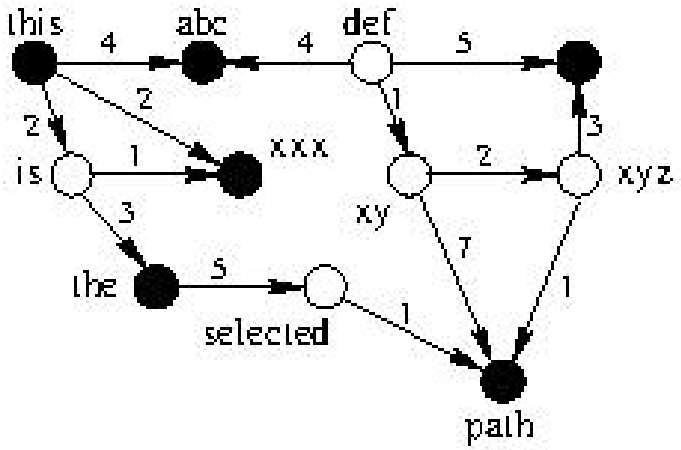
which receives the array storing the `N` pointers to those BSTs and displays, on standard output and in any order, all strings that **appear in all** BSTs.
Write the entire function using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution
To be done.

Exercise 13 (9.0 points)
A weighted, directed, and acyclic graph $G = (V, E)$ includes n vertices. G is stored as an adjacency matrix. The edges' weights are integer values. Vertices are identified by strings (of undefined length) and possess a unique attribute, i.e., their color. Each vertex can be either WHITE or BLACK (defined as constants or with an enumeration type).
Write the function

```
void longest_weight_path (int **g, int *color, char **vertex_id, int n);
```

which displays the identifiers of the vertices belonging to the path whose sum of the weights of its edges is maximum, and it is composed by vertices of alternate color, e.g., WHITE, BLACK, WHITE, BLACK, etc., or BLACK, WHITE, BLACK, WHITE, etc.
For example, given the following graph



the path with the maximum weight, including edges of alternate color, is the one including the nodes with identifiers "this", "is", "the", "selected", "path", edges with weights 2, 3, 5, 1, and whose total weight is 2+3+5+1 = 11. The parameter g stores the adjacency matrix of G, the array color reports the colors of the vertices (i.e., WHITE or BLACK), the array of strings (a 2D array) `vertex_id` stores the identifiers of the nodes, and n is the number of vertices V of the graph.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

`To be done.`